

Neural Networks Study Guide: Lecture 1 (Introduction & Foundations)

This study guide summarizes the biological foundations, artificial neuron modeling, activation functions, structural architectures (Perceptrons, Multi-Layer Feed-Forward Networks, and Recurrent Networks), logical gate realizations, and learning algorithms based on the chronological sequence of the Lecture 1 slides.

Lecture Agenda

1. **Biological Background:** Overview of biological neurons.
 2. **Artificial Neuron:** Structural elements mapping biology to computation.
 3. **Classes of Neural Networks:**
 - Perceptrons
 - Multi-Layered Feed-Forward Networks
 - Recurrent Networks
 4. **Modeling the Neuron:** Mathematical notations and weighted summation.
 5. **Activation Functions:** Step, Sign, Sigmoid, Identity, and ReLU.
 6. **Linear Separability & Logical Functions:** AND, OR, and XOR gate representations.
 7. **The Learning Cycle:** Error calculation and weight update rule.
-

1. Biological Background & ANN Ingredients

Artificial neural networks are inspired by the cell structures and communication methods of the human brain.

The Biological Neuron Components

- **Dendrites:** Branching input structures that receive incoming chemical/electrical signals from neighboring cells.
- **Cell Body (Soma):** The nucleus and center of the cell, which accumulates and processes incoming signals.
- **Axon:** A single long cable-like structure wrapped in an insulating **Myelin sheath** that carries action potentials (electrical output) away from the cell body.
- **Synapses:** Terminals at the end of axon branches that transmit the output signals to the dendrites of the next neurons.

Mapping Biology to Artificial Neural Networks (ANN)

Biological Component	Artificial Model Component	Functional Description
Dendrites	Input Features ($x_1, x_2, \dots, x_n$)	Receives features or output activations from preceding layers.
Synapses	Weights ($w_{i1}, w_{i2}, \dots, w_{in}$)	Scales the input signals, representing the connection strength.
Soma (Cell Body)	Summation Node ($\sum x_j w_{ij}$)	Aggregates all scaled inputs to calculate a total net input.
Firing Threshold	Activation Function (g)	Determines the firing output based on the input threshold.
Axon / Terminal	Output Activation (y_i)	Transmits the output activation value to the next layer.

2. Modeling & Simulation of an Artificial Neuron

An artificial neuron acts as a mathematical processing unit that computes a weighted sum of its inputs, adds a bias (or threshold), and applies an activation function.

Mathematical Definitions

- a_j : The activation value of a preceding unit j (input signal).
- $w_{j,i}$: The connection weight from unit j to the current unit i .
- in_i : The total weighted sum of inputs entering unit i :

$$in_i = \sum_j W_{j,i} a_j$$

- g : The activation function applied to the net input.
- a_i : The final activation value (output) of unit i :

$$a_i = g(in_i)$$

Artificial Neuron Flowchart

Error parsing Mermaid diagram!

Parse error on line 2:

```
...t a1"] -->|Weight W_{1,i}| Sum["Summatio
-----^
```

Expecting 'SQE', 'DOUBLECIRCLEEND', 'PE', '-)', 'STADIUMEND',
'SUBROUTINEEND', 'PIPE', 'CYLINDEREND', 'DIAMOND_STOP', 'TAGEND',

```
'TRAPEND', 'INVTRAPEND', 'UNICODE_TEXT', 'TEXT', 'TAGSTART', got  
'DIAMOND_START'
```

3. Activation Functions

Activation functions inject non-linear capabilities into neural networks. Five primary activation functions are defined in this lecture:

1. Step Function

Outputs a binary value based on a fixed threshold t .

$$\text{Step}_t(x) = \begin{cases} 1 & \text{if } x \geq t \\ 0 & \text{if } x < t \end{cases}$$

2. Sign Function

Outputs a bipolar step output symmetric around zero.

$$\text{Sign}(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

3. Sigmoid Function

A smooth, continuous, S-shaped curve bounded strictly between 0 and 1.

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

4. Identity Function

Outputs the input value directly without changes (linear activation).

$$f(x) = x$$

5. ReLU (Rectified Linear Unit)

A piecewise linear function that outputs zero for negative inputs, and acts as an identity function for positive values.

$$\text{ReLU}(x) = \max(0, x)$$

4. Classes of Neural Network Architectures

Neural networks are grouped into three structural classes based on their layers and feed-forward or recurrent connections:

Class I: Perceptrons

A Perceptron is a single-layer neural network model with inputs directly connected to the output node.

- **Formula:** Labeled mathematically as:

$$a = g(-W_0 + W_1a_1 + W_2a_2)$$

Here, $-W_0$ is the bias (also termed negative threshold), and g is a binary step activation function ($g(in) = 1$ if $in > 0$ else 0).

Class II: Multi-Layer Feed-Forward Networks

Consists of multiple layers of units where connections flow strictly forward from inputs, through hidden layers, to outputs.

- **Layers:** Contains an **Input layer**, one or more **Hidden layers**, and an **Output layer**.
- **Flow:** Output links of one layer connect only to input links of the next layer (no backwards loops or connections within the same layer).
- **Representational Capacity:** Addition of hidden layers enables multi-layer networks to represent complex, non-linear functions.
- **Example Architecture Layout (Slide 12):**
 - **Input:** 4 features (represented as input size $[N, 4]$).
 - **Hidden Layer 1:** 5 units (weights W_1 of size $[4, 5]$, activation function f_1).
 - **Hidden Layer 2:** 7 units (weights W_2 of size $[5, 7]$, activation function f_2).
 - **Output Layer:** 3 units (weights W_o of size $[7, 3]$, output size $[N, 3]$).

Class III: Recurrent Networks

A network where connection links can form loops, allowing outputs to flow back into previous layers or back into the same neuron.

- Allows the network to preserve internal states (memory), making it suitable for temporal or sequential data streams (such as speech or handwriting recognition).

5. Logical Gate Realization & Linear Separability

Single-layer perceptrons can model basic logical gates, but their capability is limited by the linear separability of the logical functions.

Truth Tables of Logic Gates

Input A	Input B	AND Output	OR Output	XOR Output
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

1. AND Gate Implementation

- **Configuration:** Weights $w_1 = 1, w_2 = 1$, and threshold $t = 2$.
- **Function:** $Y = \text{Step}_2(x_1 \cdot 1 + x_2 \cdot 1)$.
 - $x = (1, 1) \rightarrow 1(1) + 1(1) = 2 \geq 2 \rightarrow Y = 1$
 - $x = (1, 0) \rightarrow 1(1) + 0(1) = 1 < 2 \rightarrow Y = 0$
 - $x = (0, 1) \rightarrow 0(1) + 1(1) = 1 < 2 \rightarrow Y = 0$
 - $x = (0, 0) \rightarrow 0(1) + 0(1) = 0 < 2 \rightarrow Y = 0$

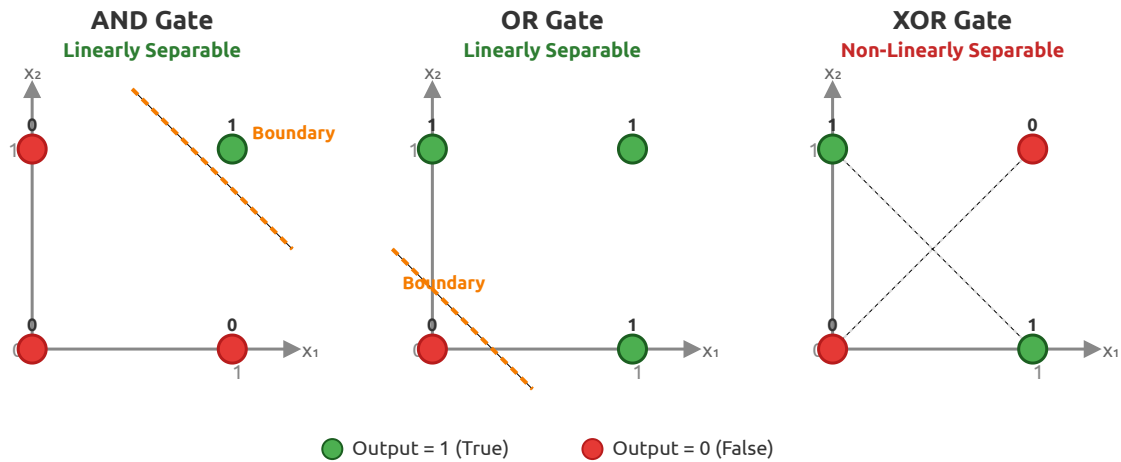
2. OR Gate Implementation

- **Configuration:** Weights $w_1 = 2, w_2 = 2$, and threshold $t = 2$.
- **Function:** $Y = \text{Step}_2(x_1 \cdot 2 + x_2 \cdot 2)$.
 - $x = (1, 1) \rightarrow 1(2) + 1(2) = 4 \geq 2 \rightarrow Y = 1$
 - $x = (1, 0) \rightarrow 1(2) + 0(2) = 2 \geq 2 \rightarrow Y = 1$
 - $x = (0, 1) \rightarrow 0(2) + 1(2) = 2 \geq 2 \rightarrow Y = 1$
 - $x = (0, 0) \rightarrow 0(2) + 0(2) = 0 < 2 \rightarrow Y = 0$

3. Linear Separability & XOR

- **Definition:** A dataset is **linearly separable** if a single straight line (or hyperplane in higher dimensions) can perfectly separate the positive output classes (1) from the negative output classes (0).
- **Perceptron Limit:** A single-layer Perceptron can only represent linearly separable functions (like AND and OR).
- **The XOR Problem:** The XOR gate outputs 1 for opposite inputs and 0 for identical inputs. Plotting these classes in a 2D grid puts the 1s and 0s diagonally opposite each other, making it impossible to separate them with a single straight line.

Linear Separability Visual Comparison



6. The Learning Process

Training a neural network involves an iterative feedback loop where weights are adjusted dynamically based on output errors.

Pseudocode Algorithm (Slide 19)

```

While epoch produces an error:
    Present network with next inputs from epoch
    Error = Target (T) - Output (O)
    If Error != 0 then:
         $W_j = W_j + LR * I_j * Error$ 
    End If
End While
  
```

Component Breakdown

- **epoch:** A single complete pass over all training datasets through the neural network.
- **Target (T):** The ground-truth (actual) expected output value.
- **Output (O):** The current predicted output generated by the network.
- **Error (Err):** The difference between target and prediction ($T - O$).
- **LR (Learning Rate):** A small positive scalar coefficient controlling the adjustment step size.
- I_j : The input value of feature j .
- W_j : The connection weight associated with input feature j .
- **Weight Update Formula:**

$$W_{j,\text{new}} = W_{j,\text{old}} + \alpha \cdot \text{Input}_j \cdot \text{Error}$$

where α is the learning rate.